

Start After Code Is Opened in VS Code

Flipkart Clone developer startup guide

Flipkart Clone Microservices Workspace

Start After Code Is Opened in VS Code

This document explains exactly what to do after you have the full Flipkart Clone code opened in **Visual Studio Code**.

Workspace path:

```
/home/user/flipkart-clone
```

If you downloaded the ZIP, extract it first and open the extracted `flipkart-clone` folder in VS Code.

PART 1: Required Software

Install these on your computer:

Tool	Required For
VS Code	Editing and running the project
Node.js 18 or newer	Local development and frontend build
Docker Desktop / Docker Engine	Running MongoDB and all microservices
Docker Compose	Starting the full app with one command
Git	Version control and pushing to GitHub
AWS CLI v2	Pushing images and linking project to AWS
AWS Toolkit for VS Code	Viewing AWS resources inside VS Code
Terraform optional	Building AWS infrastructure from code

Recommended VS Code extensions:

- AWS Toolkit
- Docker
- ESLint
- Prettier
- HashiCorp Terraform
- REST Client
- GitLens

PART 2: Open the Project in VS Code

1. Open VS Code.
2. Click **File** → **Open Folder**.
3. Select the project folder:

```
/home/user/flipkart-clone
```

4. Open the integrated terminal:

```
Terminal → New Terminal
```

5. Confirm you are in the project root:

```
pwd  
ls
```

You should see:

```
api-gateway  
user-service  
product-service  
cart-service  
order-service  
payment-service  
notification-service  
frontend  
docker-compose.yml  
README.md  
HOW_TO_RUN.md
```

PART 3: First Run Using Docker Compose

This is the easiest and recommended way.

Step 3.1: Create Environment File

```
cp .env.example .env
```

Open `.env` and change `JWT_SECRET` to a long random value before real usage.

Example:

```
JWT_SECRET=my-long-random-secret-change-this  
FRONTEND_PORT=8080  
API_GATEWAY_PORT=3000  
MONGODB_PORT=27017
```

Step 3.2: Start All Services

```
docker compose up --build
```

If you want to run in background:

```
docker compose up --build -d
```

Step 3.3: Seed Products

Open a second terminal in VS Code and run:

```
docker compose exec product-service npm run seed
```

Step 3.4: Open Application

```
Frontend:    http://localhost:8080  
API Gateway: http://localhost:3000  
Health:      http://localhost:3000/health
```

Register a new user, browse products, add to cart, checkout and view orders.

PART 4: Useful Docker Commands in VS Code Terminal

Check running containers:

```
docker compose ps
```

View all logs:

```
docker compose logs -f
```

View one service log:

```
docker compose logs -f api-gateway
```

Restart one service:

```
docker compose restart product-service
```

Rebuild one service after code changes:

```
docker compose up -d --build product-service
```

Stop all services:

```
docker compose down
```

Stop and delete MongoDB data:

```
docker compose down -v
```

Run backend syntax check:

```
./scripts/check-backend-syntax.sh
```

Run frontend build check:

```
cd frontend  
npm install  
npm run build
```

PART 5: Local Development Mode Without Full Docker

Use this only if you want to edit and hot-reload individual services.

Step 5.1: Start MongoDB Only

```
docker run -d --name flipkart-dev-mongo -p 27017:27017 mongo:7
```

If container already exists:

```
docker start flipkart-dev-mongo
```

Step 5.2: Install Service Dependencies

Run this once for each service:

```
cd api-gateway && npm install
cd ../user-service && npm install
cd ../product-service && npm install
cd ../cart-service && npm install
cd ../order-service && npm install
cd ../payment-service && npm install
cd ../notification-service && npm install
cd ../frontend && npm install
```

Step 5.3: Start Services in Separate Terminals

Terminal 1:

```
cd user-service
npm run dev
```

Terminal 2:

```
cd product-service
npm run dev
```

Terminal 3:

```
cd cart-service
npm run dev
```

Terminal 4:

```
cd order-service
npm run dev
```

Terminal 5:

```
cd payment-service
npm run dev
```

Terminal 6:

```
cd notification-service
npm run dev
```

Terminal 7:

```
cd api-gateway  
npm run dev
```

Terminal 8:

```
cd frontend  
npm start
```

If React says port 3000 is already used, accept another port like 3007 because API Gateway uses 3000.

PART 6: Testing APIs from VS Code

Use terminal or REST Client extension.

Gateway health:

```
curl http://localhost:3000/health
```

Get products:

```
curl "http://localhost:3000/api/products?limit=5"
```

Register user:

```
curl -X POST http://localhost:3000/api/users/register \
-H "Content-Type: application/json" \
-d '{
  "name": "Test User",
  "email": "test@example.com",
  "password": "password123",
  "phone": "9876543210"
}'
```

Login user:

```
curl -X POST http://localhost:3000/api/users/login \
-H "Content-Type: application/json" \
-d '{
  "email": "test@example.com",
  "password": "password123"
}'
```

PART 7: Typical Developer Workflow

Use this flow every day:

1. Open VS Code.

2. Pull latest code:

```
git pull
```

3. Start app:

```
docker compose up -d
```

4. Make code changes.

5. Rebuild changed service:

```
docker compose up -d --build service-name
```

Example:

```
docker compose up -d --build product-service
```

6. Check logs:

```
docker compose logs -f product-service
```

7. Test in browser.

8. Commit changes:

```
git add .  
git commit -m "Update product service"  
git push
```

PART 8: Prepare Code for AWS Deployment

Before AWS deployment:

1. Make sure local Docker run works.
2. Seed data locally and test user/cart/order flow.
3. Build frontend successfully.
4. Push code to GitHub.
5. Create AWS ECR repositories.
6. Build and push Docker images to ECR.
7. Create ECS task definitions and services.
8. Update production environment variables.
9. Monitor ECS service logs.

Local checks:

```
./scripts/check-backend-syntax.sh  
cd frontend && npm install && npm run build
```

Docker checks:

```
docker compose up --build -d  
docker compose ps  
docker compose logs -f
```

PART 9: Common Problems and Fixes

Docker is not running

Start Docker Desktop or Docker Engine, then run:

```
docker info
```

Products are empty

Seed products:

```
docker compose exec product-service npm run seed
```

Frontend cannot call API

Check:

- API Gateway is running on port 3000.
- REACT_APP_API_URL is correct.
- Docker Compose frontend uses /api proxy.
- Browser console network tab.

Port already in use

Edit .env:

```
FRONTEND_PORT=8081  
API_GATEWAY_PORT=3007  
MONGODB_PORT=27018
```

Restart:

```
docker compose down  
docker compose up --build
```

Service keeps restarting

Check logs:

```
docker compose logs -f service-name
```

Common causes:

- Wrong MongoDB URI
- Missing environment variable
- Syntax error
- Port conflict

PART 10: Minimum Commands You Need to Remember

```
cd /home/user/flipkart-clone  
cp .env.example .env  
docker compose up --build
```

Second terminal:

```
docker compose exec product-service npm run seed
```

Open:

```
http://localhost:8080
```